

arr { 30, 25, 27, 35, 29 }

int n = arr.length;

for (int i = 0; i < n - 1; i++) {

for (int j = 0; j < n - i - 1; j++) {

if (arr[j] > arr[j+1]) {

temp = arr[j];

arr[j] = arr[j+1];

arr[j+1] = temp;

3. [15, 10, 25, 40, 30]

10, 15, 25, 40, 30

10, 15, 25, 30, 40

i=0	j=1	j=2	j=3
15 > 10	15 > 25	25 > 40	40 > 30
X	X	X	X
(10, 15)			(30, 40)

5, 25, 40, 30

i=0	j=1	j=2	j=3
10 > 15	15 > 25	25 > 40	40 > 30
X	X	X	X

Insertion Sort

[18, 20, 20, 19, 21] 18 > 20

left = 1

i = 3

arr[i] = 19

arr[0] > 19

arr[1] > 19

arr[2] > 19

arr[3] > 19

arr[4] = 19

arr[5] = 20

arr[6] = 20

arr[7] = 21

arr[8] = 22

arr[9] = 22

arr[10] = 22

arr[11] = 22

arr[12] = 22

arr[13] = 22

arr[14] = 22

arr[15] = 22

arr[16] = 22

arr[17] = 22

while (left > 0 && arr[left] > arr[i]) {

arr[left+1] = arr[left];

left--;

3

3

arr[left+1] = arr[i];

3

3

$1000 + 500 = 1500$ $1500 \div 100 = 15$ $15 \div 2 = 7$
 $1500 \div 100 = 15$ $15 \div 2 = 7$

arr = arr.length - 1 / 2

$\{ 5, 2, 3, 9, 18, 6, 32 \}$ $\leftarrow$ mid
 $\uparrow$ low $\uparrow$ high
 $\frac{0+5}{2} = 2$
 $\frac{0+1}{2} = 0$

exception f(arr, 0, arr.length - 1);

both $\rightarrow$ ~~array~~ ~~second biggest~~
 $\swarrow$ left $\searrow$ right

f(arr, low, mid) f(arr, mid+1, high)

psarr f(int arr, int low, int high) {

if (low > high) { return 0;

int mid = $\frac{low + high}{2}$;

f(arr, low, mid);

f(arr, mid+1, high);

merge(arr, low, mid, high);

merge (start, int low, int mid, int high) {

int [] merged = new int [high - low + 1];

int blue = low

int green = high

int red = 0;

while (blue <= mid && green <= high) {

if (arr[blue] < arr[green]) {

merged[red] = arr[blue];

red++;

10, 20, 30

[9, 4, 10, 56, 45, 03]

i = 1
 0 = 56
 1 = 45
 2 = 03
 3 > 10
 94 > 56
 56 > 45
 10 > 03

arr[1+1] = 0[1]
 arr[2] = 56
 arr[4] = 56
 left = 3

arr[1+1] = 0[1]

arr[2] = 56

12 | 34 | 56 | 56 | 03
 12 | 34 | 45 | 56

12 | 34 | 45 | 56

12 | 34 | 45 | 56

12 | 34 | 45 | 56

56 > 23 45 > 23 34 > 23

arr[3] = 45 arr[2] = 23

arr[1] = 23

12 | 23 | 34 | 45 | 56

merge sort

{150, 23, 9, 18, 6, 32}

{18, 6, 32}

{18, 6, 32}

{18, 6, 32}

{18, 6, 32}

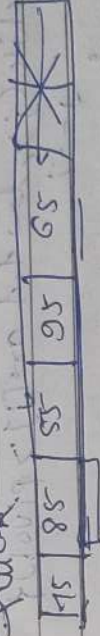

```
else {
    merged[red] = arr[green];
    red++;
    green++;
}
```

```
while (blue < mid) {
    merged[red] = arr[blue];
    red++;
    blue++;
}
```

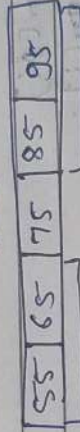
```
while (green < high) {
    merged[red] = arr[green];
    red++;
    green++;
}
```

```
for (int i = 0; i < merged.length; i++)
    arr[low+i] = merged[i];
}
```

Quick



{55, 65, 75, 85, 95}



{55, 65} {55, 95}



{65, 95}

```
if (int) (arr, int) low, int high {
```

```
    rt (low = high) {
```

```
        return;
```

```
    int pivot = pivot find (arr, low, high);
```

```
    f (arr, low, pivot - 1);
```

```
    f (arr, pivot + 1, high);
```

```
}
```


partition (int arr, int low, int high) {

int pivot = arr[low];

int i = low;

int j = high;

while (~~i < j~~ i < j) {

while (i ≤ high && arr[i] ≤ pivot) {
i++;

while (j > low && arr[j] > pivot) {
j--;

if (i < j) {

temp = arr[i];

arr[i] = arr[j];

arr[j] = temp;

int temp = arr[low]